

Mapping at scale

Using QGIS to generate hundreds of maps

Github repo:

`github.com/paulbradshaw/QGIS_param`

In 1 hour you will know:

- How to use **QGIS's Python** functionality
- How to use **parameterisation** to generate a map for every area/partner/city
- Tips on using **AI** to help with coding

Scenario:

The BBC Shared Data Unit is working on a story about flood defences. It will go to newsrooms across the UK. We want to generate a map image for each city's newsroom showing the state of flood defences in their patch.

**How do I create
a **map**?**

Do it now:

Find the shape file

Import into QGIS

Follow the guide in the repo

1. Go to [AIMS Spatial Flood Defences \(inc. standardised attributes\)](#) and download the file named `AIMS_Spatial_Flood_Defences_inc_standardised_attributes.shp.zip`
2. Unzip that file
3. Open QGIS and create a new project (*Project > New*). Save it in the same folder.
4. Select *Layer > Add Layer > Add vector layer*.
5. In the window that appears click on the ... next to the Source section (where it says *Vector Dataset(s)*) and locate the file in the downloaded folder that ends in **.shp**. Click **Add**. It should appear in the background behind the window. Click **OK** and then **Close** to return to that.
6. In the *Browser* area in the upper left side of the screen, scroll down to the XYZ Tiles section. Double-click on OpenStreetMap to add that base layer to your project (click OK to approve changing it to the same CRS). This should now appear in your Layers area.
7. Click and drag the OSM layer to move it below the flood defences layer.

Customise the appearance

1. Right-click in the flood defences layer and select **Properties...**
2. Select **Symbology** to change the colour, opacity, width, style etc. of the lines/shapes

**How do I export
an **image**?**

Do it now:

Zoom to an area

**Project > New Print
Layout...**

Create a layout (we will automate this)

1. Zoom to the part of the map you want to create an image of (e.g. zoomed on a town)
2. Go to **Project > New Print Layout...** Give it a name (e.g. cityzoom) and click **OK**
3. The layout window will appear, blank. Click the Add Map button on the left (page icon with a green plus)
4. Click and drag to draw the area where you want your map to appear.
5. Go to the **Item Properties** tab on the right side and scroll down to **Position and Size**. Alter this to any dimensions you want your image to be (change the measure to pixels if it's for web).
6. Switch to the the **Layout** Tab next to that, and scroll down to the **Resize** layout button. Click it. The image should now take up the whole canvas (you can zoom in by using the zoom buttons in the upper left corner)
7. When you are happy with the image go to **Layout > Export as Image**
8. Close the Print Layout window and return to the QGIS project



How do I do that
multiple times?

If you have to do something more than once...

Plugins > Python Console

Write/open Python in the Console **below the map.**

*Click the **Show Editor** button to open/edit .py files*

PyQGIS library ([documentation](#))

Do it now:

Download the .py file

Open it in your Console

Standardise

Python Console

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
```

>>>

Coordinate 357074,195749



Scale 1:152681



Magnifier

100%

Rotation

0.0 °



Render



EPSG:27700



(Steps in part 3 of the guide in the repo)

1. Go to the [raw file](#) on GitHub. Click File > Save As... in your browser and save the file in your project folder
2. Uncomment this line and change the path to point to your project folder

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

(To find out the path to your folder on a Mac, right click on it and select Get info. The window that appears will show you the path under 'Where:')
3. In the QGIS Python Console, open the file, then run the code


```
# Load tools that allow us to control QGIS maps and layouts from this script

from qgis.PyQt.QtCore import QRectF

from qgis.core import (
    QgsProject, QgsCoordinateReferenceSystem, QgsPrintLayout, QgsLayoutItemMap,
    QgsLayoutPoint, QgsLayoutSize, QgsUnitTypes, QgsLayoutItemLabel,
    QgsLayoutExporter, QgsRectangle, QgsCoordinateTransform, QgsPointXY
)

# Load math for handling numbers, re (regex) for text, os for file navigation

import math, re

import os
```

This is where the **output folder** is defined

Use the os library to create a folder on your computer to save the exported map images into.

It will be placed in your Downloads folder, inside a sub-folder called "qgis_images".

If the folder already exists, this does nothing – it won't overwrite anything.

```
out_dir = os.path.join(os.path.expanduser("~"), "Downloads", "qgis_images")
```

```
os.makedirs(out_dir, exist_ok=True)
```

```
out_prefix = out_dir + "/"
```

To override the folder set up above uncomment and adapt the line below

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

```
#set the Coordinate Reference System (CRS) - EPSG:4326 is used for GPS
#but 27700 is used for UK flood defences so we use this to make the map more rigorous
```

```
QgsProject.instance().setCrs(QgsCoordinateReferenceSystem("EPSG:27700"))
```

```
# Get the currently open QGIS project and store in a variable called project
```

```
project = QgsProject.instance()
```

```
#Using EPSG:27700 means we need to convert lat/long into BNG coordinates
# first set the source (EPSG:4326) and destination systems (EPSG:27700)
```

```
source_crs = QgsCoordinateReferenceSystem("EPSG:4326")
```

```
dest_crs = QgsCoordinateReferenceSystem("EPSG:27700")
```

```
#then create a transformer that uses those two
```

```
transformer = QgsCoordinateTransform(
    source_crs,
    dest_crs,
    project
)
```

#create a new print layout, in that project, store in a variable called layout

```
layout = QgsPrintLayout(project)
```

#fill it with default settings

```
layout.initializeDefaults()
```

#give it a name in QGIS

```
layout.setName("centres_layout") # Give the layout a name
```

Add a map panel to the layout – this is the rectangle that will actually show the map.

```
map_item = QgsLayoutItemMap(layout)
```

```
map_item.setFrameEnabled(True) # Draw a border around the map panel
```

Position the map panel 20mm from the left and 20mm from the top of the page.

```
map_item.attemptMove(QgsLayoutPoint(20, 20, QgsUnitTypes.LayoutMillimeters))
```

Set the map panel to be 257mm wide and 170mm tall (roughly A4 landscape minus margins).

```
map_item.attemptResize(QgsLayoutSize(257, 170, QgsUnitTypes.LayoutMillimeters))
```

```
layout.addLayoutItem(map_item) # Add the map panel to the layout page
```

Add a text label to the layout, which will show the name and coordinates of each location.

```
label = QgsLayoutItemLabel(layout)
```

```
layout.addLayoutItem(label)
```

Position the label near the top of the page (20mm from left, 5mm from top).

```
label.attemptMove(QgsLayoutPoint(20, 5, QgsUnitTypes.LayoutMillimeters))
```

def creates a function (a recipe) that we can use later. This function calculates a rectangular map area centred on a given point. It needs to be provided with four ingredients: x and y of the centre point; and half the width/height (with a default for the last item if not give).

```
def rect_around(x, y, half_width, half_height=None):
```

```
    # If no height is provided, use the same value as width
```

```
    if half_height is None:
```

```
        half_height = half_width
```

```
    # When used it will return a rectangle that can be used to set what the map is looking at.
```

```
    return QgsRectangle(
```

```
        x - half_width,
```

```
        y - half_height,
```

```
        x + half_width,
```

```
        y + half_height )
```

This is where the **locations** are stored

Create a list of locations to map. Each item has a latitude, longitude, and a name.

We start with just two items for testing, but once we get it working, we would expand it to all locations we want to create images for

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```


This is where the scale is defined

SET MAP SCALE: 1 unit on the map is 175000 in reality

MAP_SCALE = 175000

This is where the **output** is defined

```
# Set up the tool that will save the layout as an image file.
```

```
exporter = QgsLayoutExporter(layout)
```

```
settings = QgsLayoutExporter.ImageExportSettings()
```

```
# SET OUTPUT IMAGE SIZE
```

```
# 1200x795px for web use, matching the A4 layout ratio in the earlier code
```

```
settings.imageSize = QSize(1200, 795)
```

```
# Go through each location in the list, and use the function to create/export an image

for rec in listofdicstocsv:

    # Read the latitude, longitude, and name for this location.

    lat = rec.get('lat', float('nan')) # 'nan' used as a stand-in if the value is missing

    lon = rec.get('long', float('nan'))

    officialname = rec.get('officialname', 'unknown')

    # If either the latitude or longitude is missing, skip this location and move on.
    if math.isnan(lat) or math.isnan(lon):

        print(f"skipping '{officialname}': missing lat/long")

        continue

    # Move the map view so it's centred on this location, zoom using lon and lat

transformed_point = transformer.transform( QgsPointXY(lon, lat) )

# continued...
```

```
# Extract transformed coordinates

x = transformed_point.x()
y = transformed_point.y()

#Create a temporary rectangle around the point to centre it
map_item.setExtent( rect_around(x, y, 1000, 1000) )

#Set a scale to control the zoom. Each map will use that.
map_item.setScale(MAP_SCALE)

#add a label with that info
label.setText( f"{officialname} | scale 1:{MAP_SCALE:,}" )

#refresh the map before exporting
map_item.refresh()

# continued...
```

```
#give a file name - remove any tricky characters

safe_name = re.sub( r'^A-Za-z0-9._-]+', '_', officialname).strip('_')

#use that to determine the path to save to

out_path = ( f"{out_prefix}" f"{safe_name}.png" )

#and export to that path

result = exporter.exportToImage( out_path, settings )

#if it's not a success

if result != QgsLayoutExporter.Success:

    #show an error message

    raise RuntimeError(f"Export failed for {out_path}")

#print a message at the end of each loop

print("wrote", out_path)
```

How do I do that
258 times?

Recap: where the **locations** are stored

Create a list of locations to map. Each item has a latitude, longitude, and a name.

We start with just two items for testing, but once we get it working, we would expand it to all locations we want to create images for

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

Converting a CSV to a list of dicts

1. [Download the CSV of cities](#) (click the three dots in the upper right corner)
2. Type this formula in cell J2:

```
=("{ 'lat': "&B2&", 'long': "&C2&", 'officialname':  
' "&A2&" ' }")
```
3. Copy the formula down for each row. You now have a column of dicts.
4. Use this formula to join the results & put a comma between each

```
=TEXTJOIN(", ", TRUE, J:J)
```
5. In your Python code, find the opening and closing **[square brackets]** containing your list of two items, and replace those two items with the new list of dicts, keepin the square brackets.

{lat: 51.5072, 'long': -0.1275, 'officialname': 'London'}, {lat: 52.48, 'long': -1.9025, 'officialname': 'Birmingham'}, {lat: 50.8058, 'long': -1.0872, 'officialname': 'Portsmouth'}, {lat: 50.9025, 'long': -1.4042, 'officialname': 'Southampton'}, {lat: 52.9561, 'long': -1.1512, 'officialname': 'Nottingham'}, {lat: 51.4536, 'long': -2.5975, 'officialname': 'Bristol'}, {lat: 53.479, 'long': -2.2452, 'officialname': 'Manchester'}, {lat: 53.4094, 'long': -2.9785, 'officialname': 'Liverpool'}, {lat: 52.6344, 'long': -1.1319, 'officialname': 'Leicester'}, {lat: 50.8147, 'long': -0.3714, 'officialname': 'Worthing'}, {lat: 52.4081, 'long': -1.5106, 'officialname': 'Coventry'}, {lat: 54.5964, 'long': -5.93, 'officialname': 'Belfast'}, {lat: 53.8, 'long': -1.75, 'officialname': 'Bradford'}, {lat: 52.9247, 'long': -1.478, 'officialname': 'Derby'}, {lat: 50.3714, 'long': -4.1422, 'officialname': 'Plymouth'}, {lat: 51.4947, 'long': -0.1353, 'officialname': 'Westminster'}, {lat: 52.5833, 'long': -2.1333, 'officialname': 'Wolverhampton'}, {lat: 52.2304, 'long': -0.8938, 'officialname': 'Northampton'}, {lat: 52.6286, 'long': 1.2928, 'officialname': 'Norwich'}, {lat: 51.8783, 'long': -0.4147, 'officialname': 'Luton'}, {lat: 52.413, 'long': -1.778, 'officialname': 'Solihull'}, {lat: 51.544, 'long': -0.1027, 'officialname': 'Islington'}, {lat: 57.15, 'long': -2.11, 'officialname': 'Aberdeen'}, {lat: 51.3727, 'long': -0.1099, 'officialname': 'Croydon'}, {lat: 50.72, 'long': -1.88, 'officialname': 'Bournemouth'}, {lat: 51.58, 'long': 0.49, 'officialname': 'Basildon'}, {lat: 51.272, 'long': 0.529, 'officialname': 'Maidstone'}, {lat: 51.5575, 'long': 0.0858, 'officialname': 'Ilford'}, {lat: 53.39, 'long': -2.59, 'officialname': 'Warrington'}, {lat: 51.75, 'long': -1.25, 'officialname': 'Oxford'}, {lat: 51.5836, 'long': -0.3464, 'officialname': 'Harrow'}, {lat: 52.519, 'long': -1.995, 'officialname': 'West Bromwich'}, {lat: 51.8667, 'long': -2.25, 'officialname': 'Gloucester'}, {lat: 53.96, 'long': -1.08, 'officialname': 'York'}, {lat: 53.8142, 'long': -3.0503, 'officialname': 'Blackpool'}, {lat: 53.4083, 'long': -2.1494, 'officialname': 'Stockport'}, {lat: 53.424, 'long': -2.322, 'officialname': 'Sale'}, {lat: 51.5975, 'long': -0.0681, 'officialname': 'Tottenham'}, {lat: 52.2053, 'long': 0.1192, 'officialname': 'Cambridge'}, {lat: 51.5768, 'long': 0.1801, 'officialname': 'Romford'}, {lat: 51.8917, 'long': 0.903, 'officialname': 'Colchester'}, {lat: 51.6287, 'long': -0.7482, 'officialname': 'High Wycombe'}, {lat: 54.9556, 'long': -1.6, 'officialname': 'Gateshead'}, {lat: 51.5084, 'long': -0.5881, 'officialname': 'Slough'}, {lat: 53.748, 'long': -2.482, 'officialname': 'Blackburn'}, {lat: 51.73, 'long': 0.48, 'officialname': 'Chelmsford'}, {lat: 53.61, 'long': -2.16, 'officialname': 'Rochdale'}, {lat: 53.43, 'long': -1.357, 'officialname': 'Rotherham'}, {lat: 51.584, 'long': -0.021, 'officialname': 'Walthamstow'}, {lat: 51.2667, 'long': -1.0876, 'officialname': 'Basingstoke'}, {lat: 53.483, 'long': -2.2931, 'officialname': 'Salford'}, {lat: 51.4668, 'long': -0.375, 'officialname': 'Hounslow'}, {lat: 51.5528, 'long': -0.2979, 'officialname': 'Wembley'}, {lat: 52.1911, 'long': -2.2206, 'officialname': 'Worcester'}, {lat: 51.4928, 'long': -0.2229, 'officialname': 'Hammersmith'}, {lat: 51.5864, 'long': 0.6049, 'officialname': 'Rayleigh'}, {lat: 51.7526, 'long': -0.4692, 'officialname': 'Hemel Hempstead'}, {lat: 51.38, 'long': -2.36, 'officialname': 'Bath'}, {lat: 51.5127, 'long': -0.4211, 'officialname': 'Hayes'}, {lat: 54.527, 'long': -1.5526, 'officialname': 'Darlington'}, {lat: 50.8352, 'long': -0.1758, 'officialname': 'Hove'}, {lat: 50.85, 'long': 0.57, 'officialname': 'Hastings'}, {lat: 51.655, 'long': -0.3957, 'officialname': 'Watford'}, {lat: 51.9017, 'long': -0.2019, 'officialname': 'Stevenage'}, {lat: 54.69, 'long': -1.21, 'officialname': 'Hartlepool'}, {lat: 53.19, 'long': -2.89, 'officialname': 'Chester'}, {lat: 51.4828, 'long': -0.195, 'officialname': 'Fulham'}, {lat: 52.523, 'long': -1.468, 'officialname': 'Nuneaton'}, {lat: 51.5175, 'long': -0.2988, 'officialname': 'Ealing'}, {lat: 51.8168, 'long': -0.8124, 'officialname': 'Aylesbury'}, {lat: 51.6154, 'long': -0.0708, 'officialname': 'Edmonton'}, {lat: 51.755, 'long': -0.336, 'officialname': 'Saint Albans'}, {lat: 53.789, 'long': -2.248, 'officialname': 'Burnley'}, {lat: 53.7167, 'long': -1.6356, 'officialname': 'Bately'}, {lat: 53.5809, 'long': -0.6502, 'officialname': 'Scunthorpe'}, {lat: 52.508, 'long': -2.089, 'officialname': 'Dudley'}, {lat: 51.4575, 'long': -0.1175, 'officialname': 'Brixton'}, {lat: 51.5111, 'long': -0.3756, 'officialname': 'Southall'}, {lat: 55.8456, 'long': -4.4239, 'officialname': 'Paisley'}, {lat: 51.37, 'long': 0.52, 'officialname': 'Chatham'}, {lat: 51.5323, 'long': 0.0554, 'officialname': 'East Ham'}, {lat: 51.346, 'long': -2.977, 'officialname': 'Weston-super-Mare'}, {lat: 54.8947, 'long': -2.9364, 'officialname': 'Carlisle'}, {lat: 54.995, 'long': -1.43, 'officialname': 'South Shields'}, {lat: 55.7644, 'long': -4.1769, 'officialname': 'East Kilbride'}, {lat: 52.8019, 'long': -1.6367, 'officialname': 'Burton upon Trent'}, {lat: 53.9919, 'long': -1.5378, 'officialname': 'Harrogate'}, {lat: 53.099, 'long': -2.44, 'officialname': 'Crewe'}, {lat: 52.48, 'long': 1.75, 'officialname': 'Lowestoft'}, {lat: 52.37, 'long': -1.26, 'officialname': 'Rugby'}, {lat: 51.623, 'long': 0.009, 'officialname': 'Chingford'}, {lat: 51.5404, 'long': -0.4778, 'officialname': 'Uxbridge'}, {lat: 52.58, 'long': -1.98, 'officialname': 'Walsall'}, {lat: 51.475, 'long': 0.33, 'officialname': 'Grays'}, {lat: 51.3868, 'long': -0.4133, 'officialname': 'Walton upon Thames'}, {lat: 51.4002, 'long': -0.1086, 'officialname': 'Thornton Heath'}, {lat: 51.599, 'long': -0.187, 'officialname': 'Finchley'}, {lat: 51.5, 'long': -0.19, 'officialname': 'Kensington'}, {lat: 52.974, 'long': -0.0214, 'officialname': 'Boston'}, {lat: 50.4353, 'long': -3.5625, 'officialname': 'Paignton'}, {lat: 50.88, 'long': -1.03, 'officialname': 'Waterlooville'}, {lat: 53.875, 'long': -1.706, 'officialname': 'Guiseley'}, {lat: 51.5565, 'long': 0.2128, 'officialname': 'Hornchurch'}, {lat: 51.4009, 'long': -0.1517, 'officialname': 'Mitcham'}, {lat: 51.4496, 'long': -0.4089, 'officialname': 'Feltham'}, {lat: 52.4575, 'long': -2.1479, 'officialname': 'Stourbridge'}, {lat: 51.375, 'long': 0.5, 'officialname': 'Rochester'}, {lat: 53.691, 'long': -1.633, 'officialname': 'Dewsbury'}, {lat: 51.5135, 'long': -0.2707, 'officialname': 'Acton'}, {lat: 51.4449, 'long': -0.337, 'officialname': 'Twickenham'}, {lat: 53.046, 'long': -2.993, 'officialname': 'Wrecsam'}, {lat: 53.279, 'long': -2.897, 'officialname': 'Ellesmere Port'}, {lat: 54.66, 'long': -5.67, 'officialname': 'Bangor'}, {lat: 51.019, 'long': -3.1, 'officialname': 'Taunton'}, {lat: 52.7725, 'long': -1.2078, 'officialname': 'Loughborough'}, {lat: 51.54, 'long': 0.08, 'officialname': 'Barking'}, {lat: 51.6185, 'long': -0.2729, 'officialname': 'Edgware'}, {lat: 50.8094, 'long': -0.5409, 'officialname': 'Littlehampton'}, {lat: 51.576, 'long': -0.433, 'officialname': 'Ruislip'}, {lat: 51.4279, 'long': -0.1235, 'officialname': 'Streatham'}, {lat: 51.132, 'long': 0.263, 'officialname': 'Royal Tunbridge Wells'}, {lat: 53.35, 'long': -3.003, 'officialname': 'Bebington'}, {lat: 53.25, 'long': -2.13, 'officialname': 'Macclesfield'}, {lat: 52.3028, 'long': -0.6944, 'officialname': 'Wellingborough'}, {lat: 52.3931, 'long': -0.7229, 'officialname': 'Kettering'}, {lat: 51.878, 'long': 0.55, 'officialname': 'Braintree'}, {lat: 52.2919, 'long': -1.5358, 'officialname': 'Royal Leamington Spa'}, {lat: 54.1108, 'long': -3.2261, 'officialname': 'Barrow in Furness'}, {lat: 56.0719, 'long': -3.4393, 'officialname': 'Dunfermline'}, {lat: 53.3838, 'long': -2.3547, 'officialname': 'Altrincham'}, {lat: 54.0489, 'long': -2.8014, 'officialname': 'Lancaster'}, {lat: 53.4872, 'long': -3.0343, 'officialname': 'Crosby'}, {lat: 53.4457, 'long': -2.9891, 'officialname': 'Bootle'}, {lat: 51.5423, 'long': -0.0026, 'officialname': 'Stratford'}, {lat: 51.0792, 'long': 1.1794, 'officialname': 'Folkestone'}, {lat: 55.945, 'long': -3.994, 'officialname': 'Cumbarnauld'}, {lat: 51.208, 'long': -1.48, 'officialname': 'Andover'}, {lat: 51.66, 'long': -3.81, 'officialname': 'Neath'}, {lat: 52.488, 'long': -2.05, 'officialname': 'Rowley Regis'}, {lat: 54.2825, 'long': -0.4, 'officialname': 'Scarborough'}, {lat: 55.98, 'long': -3.17, 'officialname': 'Leith'}, {lat: 50.9452, 'long': -2.637, 'officialname': 'Yeovil'}, {lat: 51.451, 'long': 0.052, 'officialname': 'Eltham'}, {lat: 51.5541, 'long': -0.1744, 'officialname': 'Hamstead'}, {lat: 53.125, 'long': -1.261, 'officialname': 'Sutton in Ashfield'}, {lat: 51.4015, 'long': -0.1949, 'officialname': 'Morden'}, {lat: 51.6444, 'long': -0.1997, 'officialname': 'Barnet'}, {lat: 53.4466, 'long': -2.3086, 'officialname': 'Stretford'}, {lat: 51.408, 'long': -0.022, 'officialname': 'Beckenham'}, {lat: 51.5299, 'long': -0.3488, 'officialname': 'Greenford'}, {lat: 51.702, 'long': -0.035, 'officialname': 'Cheshunt'}, {lat: 53.48, 'long': -2.89, 'officialname': 'Kirkby'}, {lat: 51.07, 'long': -1.79, 'officialname': 'Salisbury'}, {lat: 53.4897, 'long': -2.0952, 'officialname': 'Ashton'}, {lat: 51.394, 'long': -0.307, 'officialname': 'Surbiton'}, {lat: 53.716, 'long': -1.356, 'officialname': 'Castleford'}, {lat: 51.5452, 'long': -0.0207, 'officialname': 'Catford'}, {lat: 53.3042, 'long': -1.1244, 'officialname': 'Workshop'}, {lat: 53.7492, 'long': -1.6023, 'officialname': 'Morley'}, {lat: 51.743, 'long': -3.378, 'officialname': 'Merthyr Tudful'}, {lat: 53.555, 'long': -2.187, 'officialname': 'Middlesbrough'}, {lat: 51.2834, 'long': -0.8456, 'officialname': 'Fleet'}, {lat: 50.85, 'long': -1.18, 'officialname': 'Fareham'}, {lat: 53.4487, 'long': -2.3747, 'officialname': 'Urmston'}, {lat: 51.3656, 'long': -0.1963, 'officialname': 'Sutton'}, {lat: 51.578, 'long': -3.218, 'officialname': 'Caerphilly'}, {lat: 51.128, 'long': -2.993, 'officialname': 'Bridgwater'}, {lat: 51.401, 'long': -1.323, 'officialname': 'Newbury'}, {lat: 51.4594, 'long': 0.1097, 'officialname': 'Welling'}, {lat: 51.46, 'long': -2.505, 'officialname': 'Kingswood'}, {lat: 51.886, 'long': -0.521, 'officialname': 'Dunstable'}, {lat: 51.336, 'long': 1.416, 'officialname': 'Ramsgate'}, {lat: 51.393, 'long': 0.478, 'officialname': 'Strood'}, {lat: 53.5533, 'long': -0.0215, 'officialname': 'Cleethorpes'}, {lat: 51.5932, 'long': -0.3894, 'officialname': 'Pinner'}, {lat: 52.606, 'long': 1.729, 'officialname': 'Great Yarmouth'}, {lat: 52.9711, 'long': -1.3092, 'officialname': 'Ilkeston'}, {lat: 53.653, 'long': -2.632, 'officialname': 'Chorley'}, {lat: 51.37, 'long': 1.13, 'officialname': 'Heme Bay'}, {lat: 51.872, 'long': 0.1725, 'officialname': 'Bishops Cleeve'}, {lat: 53.005, 'long': -1.127, 'officialname': 'Arnold'}, {lat: 52.724, 'long': -1.369, 'officialname': 'Coalville'}, {lat: 51.994, 'long': -0.732, 'officialname': 'Bletchley'}, {lat: 51.9165, 'long': -0.6617, 'officialname': 'Leighton Buzzard'}, {lat: 55.86, 'long': -3.98, 'officialname': 'Airdrie'}, {lat: 55.126, 'long': -1.514, 'officialname': 'Blyth'}, {lat: 51.574, 'long': 0.4181, 'officialname': 'Laindon'}, {lat: 51.684, 'long': -4.163, 'officialname': 'Llanelli'}, {lat: 52.927, 'long': -1.215, 'officialname': 'Beeston'}, {lat: 52.4629, 'long': -1.8542, 'officialname': 'Small Heath'}, {lat: 55.0456, 'long': -1.4443, 'officialname': 'Whitley Bay'}, {lat: 53.4554, 'long': -2.1122, 'officialname': 'Denton'}, {lat: 52.932, 'long': -1.127, 'officialname': 'West Bridgford'}, {lat: 51.6578, 'long': -0.2722, 'officialname': 'Borehamwood'}, {lat: 56.0011, 'long': -3.7835, 'officialname': 'Falkirk'}, {lat: 53.5239, 'long': -2.3991, 'officialname': 'Walkden'}, {lat: 51.5878, 'long': -0.3086, 'officialname': 'Kenton'}, {lat: 54.0819, 'long': -0.1923, 'officialname': 'Bridlington'}, {lat: 54.61, 'long': -1.27, 'officialname': 'Billingham'}, {lat: 52.918, 'long': -0.638, 'officialname': 'Grantham'}, {lat: 55.0097, 'long': -1.4448, 'officialname': 'North Shields'}, {lat: 51.947, 'long': -0.283, 'officialname': 'Hitchin'}, {lat: 52.7858, 'long': -0.1529, 'officialname': 'Spalding'}, {lat: 51.36, 'long': 0.61, 'officialname': 'Rainham'}, {lat: 51.978, 'long': -0.23, 'officialname': 'Letchworth'}, {lat: 51.6114, 'long': 0.5207, 'officialname': 'Wickford'}, {lat: 53.4111, 'long': -2.8403, 'officialname': 'Huyton'}, {lat: 51.6717, 'long': -1.2783, 'officialname': 'Abingdon'}, {lat: 51.32, 'long': -2.208, 'officialname': 'Trowbridge'}, {lat: 52.5812, 'long': -1.093, 'officialname': 'Wigston Magna'}, {lat: 51.606, 'long': -1.241, 'officialname': 'Didcot'}, {lat: 51.433, 'long': -0.933, 'officialname': 'Earley'}, {lat: 51.459, 'long': 0.138, 'officialname': 'Bexleyheath'}, {lat: 53.4429, 'long': -1.4698, 'officialname': 'Ecclesfield'}, {lat: 53.698, 'long': -2.461, 'officialname': 'Darwen'}, {lat: 53.5333, 'long': -2.2833, 'officialname': 'Prestwich'}, {lat: 51.602, 'long': -3.342, 'officialname': 'Pontypridd'}, {lat: 55.828, 'long': -4.214, 'officialname': 'Rutherglen'}, {lat: 51.1295, 'long': 1.3089, 'officialname': 'Dover'}, {lat: 50.8365, 'long': -0.7792, 'officialname': 'Chichester'}, {lat: 51.2226, 'long': 1.4006, 'officialname': 'Deal'}, {lat: 51.9, 'long': -1.15, 'officialname': 'Bicester'}, {lat: 51.5467, 'long': -0.37, 'officialname': 'Northolt'}, {lat: 55.7742, 'long': -3.9183, 'officialname': 'Wishaw'}, {lat: 51.3652, 'long': -0.1676, 'officialname': 'Carshalton'}, {lat: 53.001, 'long': -1.197, 'officialname': 'Bulwell'}, {lat: 54.591, 'long': -5.68, 'officialname': 'Newtownards'}, {lat: 54.326, 'long': -2.745, 'officialname': 'Kendal'}, {lat: 55.082, 'long': -1.585, 'officialname': 'Cramlington'}, {lat: 52.3353, 'long': -2.0579, 'officialname': 'Bromsgrove'}, {lat: 51.703, 'long': -3.041, 'officialname': 'Pont-y-pŷll'}, {lat: 51.509, 'long': -0.338, 'officialname': 'Hanwell'}, {lat: 51.2279, 'long': -2.3215, 'officialname': 'Frome'}, {lat: 51.5981, 'long': -0.1149, 'officialname': 'Wood Green'}, {lat: 52.5708, 'long': -2.0457, 'officialname': 'Darlaston'}, {lat: 55.181, 'long': -1.568, 'officialname': 'Ashington'}, {lat: 52.9877, 'long': -2.1327, 'officialname': 'Longton'}, {lat: 52.7661, 'long': -0.886, 'officialname': 'Melton Mowbray'}, {lat: 52.606, 'long': -1.9179, 'officialname': 'Aldridge'}, {lat: 53.5452, 'long': -2.3999, 'officialname': 'Farnworth'}, {lat: 51.552, 'long': -0.097, 'officialname': 'Highbury'}, {lat: 53.3761, 'long': -2.1897, 'officialname': 'Cheadle Hulme'}, {lat: 54.62, 'long': -1.58, 'officialname': 'Newton Aycliffe'}, {lat: 52.4299, 'long': -1.9355, 'officialname': 'Bournville'}, {lat: 52.009, 'long': -0.789, 'officialname': 'Shenley Brook End'}, {lat: 54.85, 'long': -1.83, 'officialname': 'Consett'}, {lat: 51.3211, 'long': -0.1386, 'officialname': 'Coulson'}, {lat: 52.566, 'long': -2.0728, 'officialname': 'Bilston'}, {lat: 52.7001, 'long': -2.5157, 'officialname': 'Wellington'}, {lat: 54.663, 'long': -1.676, 'officialname': 'Bishop Auckland'}, {lat: 52.395, 'long': -1.979, 'officialname': 'Longbridge'}, {lat: 52.614, 'long': -2.004, 'officialname': 'Bloxwich'}, {lat: 51.5557, 'long': 0.2512, 'officialname': 'Upminster'}, {lat: 53.321, 'long': -3.48, 'officialname': 'Rhyll'}, {lat: 52.267, 'long': -2.153, 'officialname': 'Droitwich'}, {lat: 53.5355, 'long': -2.5658, 'officialname': 'Hindley'}, {lat: 53.549, 'long': -2.529, 'officialname': 'Westhoughton'}, {lat: 51.3589, 'long': 1.4394, 'officialname': 'Broadstairs'}

Use a slice in the for loop to test it:

```
# Adding [10:20] after the list name means it will only loop through items in positions 10 to 19
for rec in listofdicstocsv[10:20]:
    # Read the latitude, longitude, and name for this location.

    lat = float(rec.get('lat', float('nan')))    # 'nan' used as a stand-in if the value is missing
    lon = float(rec.get('long', float('nan')))

    officialname = rec.get('officialname', 'unknown')

    # If either the latitude or longitude is missing, skip this location and move on.
    if math.isnan(lat) or math.isnan(lon):
        print(f"skipping '{officialname}': missing lat/long")
        continue
```

**How do I do that
for different size
areas?**

**We need different
scales:**

**Large conurbation,
smaller area**

Vibe coding time?

Use meta prompting to ask AI to design the prompt first

Include:

- The context (journalism)
- The objective
- Detailed instructions (specific names, files etc.)

You are my mentor, a data journalist with over a decade's experience in the field. You have advanced **statistical knowledge** as well as a healthy **scepticism** when dealing with both data and human sources.

I will ask you for help with some coding - you are happy to guide me, but **you don't want me to become deskilled and too reliant** on you, so your advice will always be designed to **force me to think for myself, learn new skills and concepts, and practise** those.

When you provide code **add comments explaining each step** as if to a person with no coding experience.

I prefer simple code that requires multiple lines to complex code in one or two lines.

Flag any assumptions you are making about the data or my question. Warn me if the question does not contain enough information to answer accurately, or if the result could be misleading without additional context.

Attached is some Python that generates images for maps centred at hundreds of locations. **Adapt this so that it now generates those images at two different scales.**

Comment the code to highlight the section of code where it does this, so that I can tweak it and try different zoom levels.

Parameters

https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_tw_oSize.py

The section setting **scale** parameters

```
# SET MAP SCALES:
```

```
MAP_SCALES = [  
    175000,    # original scale  
    120000    # zoomed-in version  
]
```

The section using those parameters

```
# NEW: LOOP THROUGH MULTIPLE SCALES
```

```
for MAP_SCALE in MAP_SCALES:
```

```
    #Create a temporary rectangle around the point to centre it
```

```
    map_item.setExtent(
```

```
        rect_around(x, y, 1000, 1000)
```

```
)
```

```
    #Apply the current scale
```

```
    map_item.setScale(MAP_SCALE)
```

```
    #add a label with that info
```

```
    label.setText(f"{officialname} | scale 1:{MAP_SCALE:,}")
```

...and adding to the filename

```
#use that to determine the path to save to
```

```
out_path = (
```

```
    f"{out_prefix}"
```

```
    f"{safe_name}_{MAP_SCALE}.png" # NEW: ADD SCALE TO FILENAME
```

```
)
```

The critical friend:
What AI should tell you

Things to consider

- Converting between the CRS of base map vs shapefile
- Converting between layout size (mm) and imageSize (px)
- Exporting for print/200dpi resolution
- Scale categories instead of raw numbers (and more than two)

Try this:
**Other optional files in
the repo**

Key takeaways

- If you have to do it more than once...
- Vibe coding is fun but *mentor* coding is more rewarding

Extra resources:

- https://github.com/paulbradshaw/QGIS_param
- Tim Green: [Vibe Coding Threatens Journalism: Why Newsrooms Need Governance Now](#)
- [FT newsletter](#) on their methodology using AI to assist with coding in mapping flood risk areas